

EWAT

Early Warning and Anomaly Typing
dans les architectures microservices Kubernetes

Rapport de stage

Devoteam — 2026

Wassim Badraoui

EPITA — Promotion 2026
`wassim.badraoui@epita.fr`

Résumé

Les architectures microservices Kubernetes génèrent un flux permanent de changements bénins — déploiements progressifs, autoscaling, reconfigurations — que les systèmes de détection d’anomalies classiques confondent avec de vraies pannes, produisant un taux de fausses alarmes de 100 % sur les drifts bénins quel que soit le seuil. Ce rapport présente EWAT (*Early Warning and Anomaly Typing*), un pipeline de détection précoce et de typage automatique des anomalies en quatre étapes : détection de drift par MMD via Random Fourier Features avec mécanisme look-through, encodage spatio-temporel par réseau STGCN, typage contrastif siamois avec clustering hiérarchique, et classifieurs de précurseurs typés par type de cluster. Évalué sur ewat_v3 (299 épisodes, 15 scénarios, cluster Kubernetes réel), le pipeline valide deux hypothèses principales : H1 (silhouette test = 0.519 ± 0.092 sur 5 graines, les embeddings sont structurables) et H3 (AUROC = 0.973 ± 0.012 , lead time opérationnel de 3 min). Il réduit les fausses alarmes sur drifts de 100 % à 8.3 % par rapport à la baseline z-score tout en maintenant un lead time de 3.0 min. L’hypothèse H2a (look-through) est réfutée ($p = 0.27$) pour une raison identifiée et quantifiée : les épisodes trop courts (≈ 21 steps) ne laissent pas de fenêtre utile suffisante au mécanisme de confirmation temporelle, motivant la collecte ewat_v4 avec des épisodes ≥ 40 steps.

Mots-clés : détection d’anomalies, early warning, microservices, Kubernetes, STGCN, Transfer Entropy, drift de concept, typage contrastif.

Table des matières

1	Introduction	3
1.1	Contexte	3
1.2	Problème	3
1.3	Contributions	4
1.4	Plan du document	4
2	Formalisation du problème	6
2.1	Graphe de services	6
2.2	Signal de télémétrie	6
2.3	Régimes opérationnels	7
2.4	Pipeline EWAT	7
2.5	Hypothèses et critères de falsification	8
3	Dataset ewat_v3	9
3.1	Infrastructure expérimentale	9
3.2	Protocole de collecte	9
3.3	Composition	10
3.4	Qualité du signal	10
3.5	Limitations	10
4	Implémentation du pipeline	12
4.1	Étape 0 — Détection de drift (MMD-RFF)	12
4.2	Étape 1 — Encodeur STGCN	13
4.3	Étape 2 — Typage siamois	13
4.4	Étape 2b — Ontologie	13
4.5	Étape 3 — Précurseurs typés	14
4.6	Alertes — AlertAssembler	14
5	Résultats	15
5.1	H1 — Structurabilité des embeddings	15
5.2	H2a — Séparabilité du drift par look-through	16
5.3	H2b — Identification du régime $\theta_{\text{drift} \cap \text{anomaly}}$	17
5.4	H3 — Prédicibilité des précurseurs typés	18
5.5	Système d’alerte — comparaison avec le z-score	19
6	Expériences complémentaires	21
6.1	Ablation des modalités — H1 (réentraînement)	21
6.2	Ablation features — H3 (masquage à l’inférence)	22

6.3	Comparaison d’architectures d’encodeur	23
6.4	Ontologie — relations causales service-level	24
6.5	Transfert — RCAEval RE2-OB	25
6.5.1	Zero-shot	26
6.5.2	Few-shot — Stratégie A	26
7	Discussion	28
7.1	Résultats négatifs comme contribution	28
7.2	Apport EWAT vis-à-vis du z-score	29
7.3	Valeur de l’encodeur STGCN	29
7.4	Limitations	30
7.5	Perspectives — ewat_v4	31
8	Difficultés rencontrées et évolutions méthodologiques	32
8.1	Fuite d’information inter-splits (correction H1/H3)	32
8.2	Bug LayerNorm TCN (correction silencieuse)	32
8.3	Méthode d’explicabilité gradient invalidée	33
8.4	Biais écologique dans la Transfer Entropy cluster-level	34
8.5	Résultats inattendus sur les baselines	34
8.6	Collecte de données : pannes d’infrastructure	35
8.7	Transfert de domaine : hypothèse initiale invalidée	35
	Conclusion	37
	Références	37

1 Introduction

Les architectures microservices déployées sur Kubernetes sont devenues la norme dans les organisations qui adoptent une cadence de livraison continue. Chaque service peut être mis à jour, mis à l'échelle ou reconfiguré indépendamment, ce qui produit un flux permanent de changements bénins dans le signal d'observabilité : déploiements progressifs, autoscaling horizontal, modifications de quota. Les outils de détection d'anomalies classiques — fondés sur des seuils statiques ou des z-scores univariés [1] — ne distinguent pas ces drifts bénins des vraies pannes. Il en résulte un niveau élevé de faux positifs en production, qui épuise la confiance des équipes d'exploitation et retarde la réaction aux incidents réels.

EWAT (*Early Warning and Anomaly Typing*) est un pipeline de détection précoce et de typage automatique des anomalies conçu pour répondre à ce problème. Son positionnement est explicitement distinct du RCA (*Root Cause Analysis*) [2] : là où le RCA est post-mortem (où, pourquoi, après la panne), EWAT est un système d'alerte précoce (quoi, dans combien de temps, avant la panne). L'objectif n'est pas d'identifier la cause racine, mais de détecter les signes précurseurs d'une dégradation imminente et d'en caractériser le type pour orienter la réaction opérationnelle.

1.1 Contexte

Ce stage s'est déroulé chez Devoteam, société de conseil spécialisée en transformation numérique, dans un contexte de montée en puissance de l'observabilité des plateformes cloud. Le cluster expérimental `observit-cluster1` (RKE2 v1.32, 9 nœuds) héberge l'application *Online Boutique* de Google, une boutique en ligne de référence composée de microservices hétérogènes. La stack d'observabilité combine Prometheus pour les métriques d'infrastructure et un OpenTelemetry Collector pour les traces et logs applicatifs — deux sources complémentaires qui constituent ensemble le signal $S(t) \in \mathbb{R}^{N \times 17}$.

Les injections de pannes contrôlées sont réalisées avec Chaos Mesh, un opérateur Kubernetes qui permet de simuler des scénarios reproductibles : saturation CPU, fuite mémoire, déploiement défectueux, montée en charge soudaine. Cet outillage permet de constituer un dataset labellisé tout en conservant un cluster réel, avec ses couplages inter-services et ses bruits opérationnels — une condition indispensable pour que les résultats soient transférables en production.

1.2 Problème

Un déploiement progressif (`rolling_deploy`) et une saturation CPU (`cpu_starvation`) produisent tous deux des anomalies statistiques dans les métriques de latence et d'er-

reur. Un z-score à $\sigma = 2.5$ déclenche une alerte dans les deux cas sans distinction. Ce comportement est documenté dans la section 5.5 : quel que soit le seuil $\sigma \in [2.0, 3.5]$, le taux de fausses alarmes sur les drifts bénins reste à 100 %.

La distinction fondamentale entre drift et anomalie exige une modélisation explicite de quatre régimes opérationnels $\theta(t) \in \Theta$ plutôt que trois. Trois régimes (*normal*, *drift*, *anomalie*) sont insuffisants car ils ne modélisent pas le cas hybride $\theta_{\text{drift} \cap \text{anomaly}}$: un déploiement défectueux qui présente simultanément les signatures d’un drift bénin (changement de distribution graduel) et d’une anomalie réelle (dégradation des SLOs). Ce quatrième régime, incarné par le scénario `faulty_deploy_overlap` du dataset, requiert un mécanisme de *look-through* pour ne pas supprimer le signal d’anomalie pendant la transition de drift.

1.3 Contributions

Ce travail apporte quatre contributions principales :

1. **Formalisation à quatre régimes et pipeline EWAT *end-to-end*.** Définition formelle de $G(t)$, $S(t) \in \mathbb{R}^{N \times 17}$, des quatre régimes Θ , et d’un pipeline en quatre étapes séquentielles (Section 2). Implémentation complète avec 302 tests unitaires et lint propre (Ruff, mypy).
2. **H1 validée — structurabilité empirique des types de pannes.** Les embeddings STGCN forment des clusters bien séparés dans l’espace latent : silhouette test = 0.519 ± 0.092 sur 5 graines, minimum = $0.414 \gg 0.3$ (seuil de Kaufman & Rousseeuw [3]). $K = 10$ clusters stables, $K^* \in \{9, 10, 11\}$ selon la graine.
3. **H3 validée — précurseurs typés à horizon k^* opérationnel.** AUROC moyen = 0.973 ± 0.012 sur 5 graines, 7–8 types prédictibles sur K . Horizon optimal $k^* = 6$ steps (3 min) pour 5 types sur 8 — lead time opérationnel directement exploitable.
4. **H2 négatif exploitable — motivation de `ewat_v4`.** Le mécanisme look-through n’apporte pas de réduction significative du FPR sur anomalie ($p = 0.27$). La cause est identifiée précisément : les épisodes de ≈ 21 steps laissent ≈ 7 steps utiles après le warm-up du DriftDetector (8 steps) et la confirmation temporelle (3–6 steps). Ce résultat négatif motive la collecte `ewat_v4` avec des épisodes ≥ 40 steps.

1.4 Plan du document

La section 2 pose les bases mathématiques du problème. La section 3 décrit le dataset `ewat_v3` (299 épisodes, 15 scénarios). La section 4 détaille l’implémentation

de chaque étape du pipeline. La section 5 rapporte les résultats des hypothèses H1, H2a, H2b et H3, ainsi que la comparaison avec la baseline z-score. La section 6 présente les expériences complémentaires : ablation des modalités, comparaison d’architectures, ontologie causale et transfert de domaine sur RCAEval. La section 7 discute les apports, les limitations et les perspectives pour `ewat_v4`.

2 Formalisation du problème

Une formalisation explicite est nécessaire pour deux raisons. D’abord, elle contraint les choix d’implémentation : les règles d’agrégation intra-service, le nombre de régimes $|\Theta|$ et le budget de latence découlent directement des définitions formelles. Ensuite, elle définit les critères de falsification des hypothèses de manière non ambiguë, ce qui rend les résultats négatifs aussi informatifs que les résultats positifs.

2.1 Graphe de services

Soit $G(t) = (V, E(t), w_E(t))$ le graphe de services à l’instant t , où V est l’ensemble des services Kubernetes (Deployments), constant avec $|V| = N = 6$, et $E(t)$ les arêtes pondérées par $w_E : E(t) \rightarrow \mathbb{R}^3$, $e_{ij}(t) \mapsto (\text{volume}_{ij}, \text{latence_med}_{ij}, \text{taux_erreur}_{ij})$.

Une arête e_{ij} est présente à l’instant t si et seulement si le volume de requêtes entre le service i et le service j est strictement positif sur la fenêtre glissante précédente. Les poids sont agrégés depuis les spans OTel selon les règles suivantes, choisies pour respecter les propriétés statistiques de chaque type de mesure :

- Saturation (CPU, RAM, `net_sat`, `disk_io`) : **maximum** sur les pods du service — la saturation la plus haute détermine le comportement.
- Taux (`error_rate`, `log_warn_rate`) : **somme pondérée par volume** — un service à faible trafic et taux élevé ne doit pas être dilué.
- Latence (`latency_p99`, `span_dur_median`) : **percentile 99 sur l’union** des distributions individuelles des pods — jamais un percentile de percentiles, qui sous-estime les queues de distribution [4].
- Métriques structurelles (`trace_depth`, `fan_out`, `lexical_entropy`) : **médiane** — robuste aux outliers transitoires.

2.2 Signal de télémétrie

Le signal $S(t) \in \mathbb{R}^{N \times 17}$ est défini comme la concaténation de trois modalités :

$$S(t) = [M(t) \mid T(t) \mid L(t)]$$

avec :

- $M(t) \in \mathbb{R}^{N \times 7}$ — métriques Prometheus, couvrant les ressources selon la méthodologie USE [4] (utilisation, saturation, erreurs) : `cpu_util`, `ram_util`, `latency_p99`, `error_rate_http`, `net_sat`, `disk_io`, `queue_depth`.
- $T(t) \in \mathbb{R}^{N \times 6}$ — traces OTel, capturant la structure et la qualité des appels inter-services : `span_dur_median`, `abnormal_span_rate`, `trace_depth`, `fan_out`, `retry_rate`, `latency_cv`.

- $L(t) \in \mathbb{R}^{N \times 4}$ — logs OTel, dont deux features sémantiques : `log_error_rate`, `log_warn_rate`, `semantic_anomaly` (distance SentenceBERT au centroïde normal), `lexical_entropy`.

La modalité $M(t)$ fournit les signaux de ressource matures et stables ; $T(t)$ capture les effets de propagation dans le graphe d’appel ; $L(t)$ enrichit avec une sémantique applicative non disponible dans les métriques brutes. L’ablation par réentraînement (Section 6.1) montre que $M(t)$ seul suffit pour la structuration géométrique (H1), tandis que $T(t)$ et $L(t)$ sont indispensables pour la prédictibilité des précurseurs (H3).

2.3 Régimes opérationnels

Nous définissons quatre régimes $\theta(t) \in \Theta$:

$$\Theta = \{\theta_{\text{normal}}, \theta_{\text{drift}}, \theta_{\text{anomaly}}, \theta_{\text{drift} \cap \text{anomaly}}\}$$

θ_{normal} est l’état nominal du système, sans injection. θ_{drift} correspond aux changements bénins de distribution — déploiements progressifs, autoscaling, modifications de quota — qui ne dégradent pas les SLOs. θ_{anomaly} regroupe les vrais incidents : saturation ressource, crash, fuite mémoire, latence anormale. $\theta_{\text{drift} \cap \text{anomaly}}$ modélise le cas hybride d’un déploiement défectueux : la distribution change (drift) et les SLOs se dégradent simultanément (anomalie). Ce quatrième régime exige un mécanisme de look-through dans l’étape de détection de drift, afin de ne pas supprimer le signal d’anomalie sous prétexte qu’un drift est en cours. Trois régimes seraient insuffisants : le régime $\theta_{\text{drift} \cap \text{anomaly}}$ ne peut pas être correctement géré ni comme drift pur (l’alerte serait supprimée) ni comme anomalie pure (la logique de recalibration du détecteur serait incorrecte).

2.4 Pipeline EWAT

Le pipeline se décompose en quatre étapes séquentielles, conçues pour être indépendantes et testables unitairement :

Étape 0 — Détection de drift (MMD-RFF). $\widehat{\text{MMD}}^2(W_{\text{ref}}, W_{\text{cur}})$ via Random Fourier Features avec mécanisme *look-through* [5]. La complexité est $O(nD)$ avec $D = 256$ features aléatoires, permettant une inférence en temps réel. Le seuil $\varepsilon_{\text{drift}} = 0.5226$ est calibré par critère de Youden sur les drifts bénins injectés (AUC = 0.60 sur train). Budget de latence : < 1 s.

Étape 1 — Encodeur STGCN. $z_e = \text{Enc}_{\theta}(S_{[t-W, t+\delta]}, G(t)) \in \mathbb{R}^{64}$, convolution spatio-temporelle sur le graphe de services [6]. L’encodeur joint la dimension spatiale (structure du graphe $G(t)$) et la dimension temporelle (série

$S(t)$) via un réseau TCN, produisant un embedding $z_e \in \mathbb{R}^{64}$ par épisode. Budget : < 2 s.

Étape 2 — Typage contrastif. Réseau siamois appris sur les paires de scénarios Chaos Mesh : $d_\varphi(z_i, z_j) \rightarrow 0$ si même type, $\rightarrow 1$ sinon. Le clustering hiérarchique agglomératif sur les embeddings produit K types $\{C_1, \dots, C_K\}$ sélectionnés par score de silhouette sur train ($K = 10$ sur ewat_v3). En validation et test, les labels sont assignés par plus proche centroïde depuis les centroïdes train — ce qui garantit l’absence de fuite d’information inter-splits.

Étape 2b — Ontologie. Relations temporelles ($C_i \rightarrow C_j$, support ≥ 3), causales (Transfer Entropy, estimateur KSG [7], 100 permutations, BH-FDR [8]) et de co-occurrence (χ^2 Yates, correction Holm–Bonferroni).

Étape 3 — Précurseurs typés. $\hat{p}_i(t) = f_i(S_{[t-k, t]}, G(t)) \in [0, 1]$, $k_i^* = \arg \max_k \text{AUROC}(f_i, k)$ sélectionné sur val, évalué sur test uniquement. Budget : < 1 s.

La sortie est $\text{Alert}(t) = (C_i, \hat{p}_i(t), k_i^*, \text{fiche}_{C_i})$, enrichie d’une fiche d’interprétabilité par cluster générée par permutation importance et validée par KernelSHAP.

2.5 Hypothèses et critères de falsification

Chaque hypothèse est formulée avec un critère quantitatif de falsification, de sorte qu’un résultat négatif soit aussi informatif qu’un résultat positif.

H1 — Structurabilité. Les embeddings STGCN forment des clusters exploitables dans l’espace latent, définis par silhouette ≥ 0.3 en held-out sur 5 graines $\times$ 5 splits. Le seuil 0.3 est justifié par Kaufman & Rousseeuw [3] comme limite minimale d’une structure exploitable.

H2a — Séparabilité drift par look-through. Le mécanisme look-through réduit significativement le FPR sur anomalie à rappel constant, mesuré par un test de Student unilatéral apparié ($p < 0.05$) sur les 45 épisodes test.

H2b — Identification du régime $\theta_{\text{drift} \cap \text{anomaly}}$. Le cluster C8 (faulty_deploy_overlap) présente un overlap drift + alerte significativement supérieur aux clusters de drift pur (C5, C6, C9), testé par Fisher exact ou proportion bootstrap.

H3 — Prédicibilité des précurseurs typés. L’AUROC par type est supérieur à la baseline aléatoire (0.5) pour chaque horizon k , k_i^* sélectionné sur val et évalué sur test. Les intervalles de confiance à 95 % sont estimés par bootstrap (1000 rééchantillonnages).

3 Dataset ewat_v3

Le dataset `ewat_v3` est issu d’une collecte expérimentale sur infrastructure réelle. L’utilisation d’un cluster Kubernetes de production simulée — plutôt que d’un simulateur ou d’un dataset public — est un choix délibéré : les couplages inter-services réels, les bruits d’observabilité et les artefacts de timing (retards de scraping Prometheus, buffers OTel) sont indispensables pour que le pipeline soit évaluable dans des conditions représentatives du déploiement en production.

3.1 Infrastructure expérimentale

Le cluster `observit-cluster1` est un cluster RKE2 v1.32.7 composé de 9 nœuds (8 Ready, 1 NotReady : `observit-cluster1-workers-58w74-mwxb2`, source du 16.7% de NaN sur `disk_io` pour le service `product-catalog`). L’application hébergée est *Online Boutique* de Google, une boutique en ligne constituée de microservices hétérogènes (Go, Python, Java, Node.js) couvrant les services `frontend`, `cart`, `load-generator`, `recommendation`, `ad` et `product-catalog`.

Les injections de pannes sont réalisées avec Chaos Mesh, un opérateur Kubernetes permettant de définir des scénarios reproductibles sous forme de CRDs. La stack d’observabilité combine Prometheus (métriques Kubernetes et applicatives) et un OpenTelemetry Collector recevant les traces et logs applicatifs en OTLP gRPC.

3.2 Protocole de collecte

La collecte suit un protocole en trois phases :

1. **Phase 1 — Enregistrement** (`scripts/record_episode.py`) : injection d’un scénario Chaos Mesh, dump brut des métriques Prometheus et des spans OTel sur une fenêtre de ≈ 10 min ($T \approx 21$ steps à 30 s/step).
2. **Phase 2 — Construction des features** (`scripts/build_features.py`) : extraction du signal $S(t) \in \mathbb{R}^{N \times 17}$, du graphe $G(t)$ et des labels de régime depuis les dumps bruts.
3. **Phase 3 — Assemblage** (`scripts/assemble_dataset.py`) : split stratifié 209/45/45 (train / val / test) garantissant la représentation de chaque scénario dans les trois splits.

Le split stratifié assure qu’aucun scénario n’est absent du jeu de test, condition nécessaire pour évaluer H3 par type de cluster. Le ratio 209/45/45 correspond approximativement à un split 70/15/15, classique pour des datasets de taille limitée.

3.3 Composition

Le dataset `ewat_v3` comprend **299 épisodes** (300 collectés, 1 rejeté — `network_loss_018`, Loki 100 % NaN) répartis sur 15 scénarios :

- **4 drifts bénins** : `drift_config_change`, `drift_rolling_deploy`, `drift_scale_up`, `drift_traffic_ramp`.
- **11 anomalies vraies** : `cpu_starvation`, `crash`, `fail_slow_cpu`, `fail_slow_latency`, `faulty_deploy_overlap`, `intermittent_error`, `memory_pressure`, `network_loss`, `noisy_neighbor`, `oom`, `resource_leak`.

Les 6 services instrumentés sont : `frontend`, `cart`, `load-generator`, `recommendation`, `ad`, `product-catalog`.

Chaque scénario dispose de ≈ 20 répétitions, réparties de façon équilibrée sur les trois splits. Le scénario `faulty_deploy_overlap` est particulier : il représente le régime $\theta_{\text{drift} \cap \text{anomaly}}$, avec un déploiement défectueux qui déclenche simultanément un drift de distribution et une dégradation des SLOs.

3.4 Qualité du signal

TABLE 1 – Taux de valeurs manquantes par modalité.

Feature	NaN	Cause
<code>cpu_util</code> , <code>ram_util</code> , <code>net_sat</code> , <code>queue_depth</code>	0 %	✓
<code>latency_p99</code> , <code>error_rate_http</code>	0 %	patchés depuis spans OTel
Traces (7–12)	0 %	✓
Logs (13–16)	0.4 %	résiduel irréductible
<code>disk_io</code>	16.7 %	nœud NotReady (<code>product-catalog</code>)

Le taux global de NaN est de ≈ 1.5 %, quasi-entièrement concentré sur `disk_io` pour le service `product-catalog`. Ce NaN est structurel : le nœud `observit-cluster1-workers-58w7` est resté en état NotReady pendant l’intégralité de la collecte `ewat_v3`, rendant les métriques de disque indisponibles pour ce service. Les NaN sont imputés par zéro dans le pipeline (valeur par défaut de la métrique en l’absence de scraping), ce qui sous-estime l’importance réelle de `disk_io` — confirmé par l’ablation LOO (Section 6.2, $\Delta = -0.088$).

3.5 Limitations

L1 — Épisodes trop courts (≈ 21 steps, ≈ 10.5 min). La durée a été optimisée pour maximiser le nombre d’épisodes (trade-off débit vs. profondeur temporelle). Impact : insuffisant pour le mécanisme look-through (warm-up ≈ 8 steps, confirmation 3–6 steps), et lead time maximal observable limité à ≈ 5 min. Résolu dans `ewat_v4` (≥ 40 steps, durée ≥ 20 min par épisode).

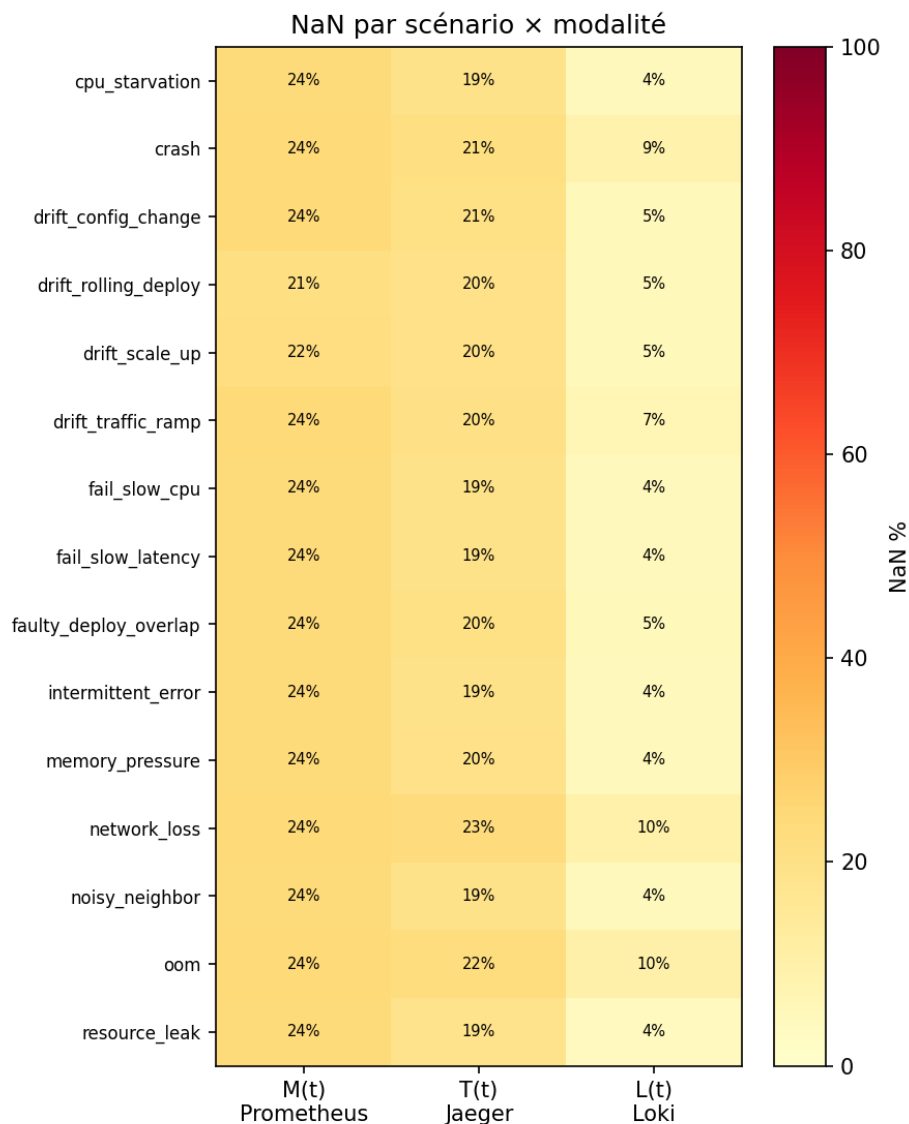


FIGURE 1 — Taux de NaN par feature et par scénario. `disk_io` pour `product-catalog` présente systématiquement 16.7 % de NaN (nœud NotReady).

L2 — `disk_io` 16.7 % NaN. L’ablation H3 (Section 6.2) montre que `disk_io` est la feature la plus critique pour les précurseurs ($\Delta = -0.088$), malgré ses valeurs manquantes — son importance réelle est probablement sous-estimée. Résolu dans `ewat_v4` par déploiement d’un OTEL SDK applicatif permettant de collecter les métriques de disque indépendamment du nœud.

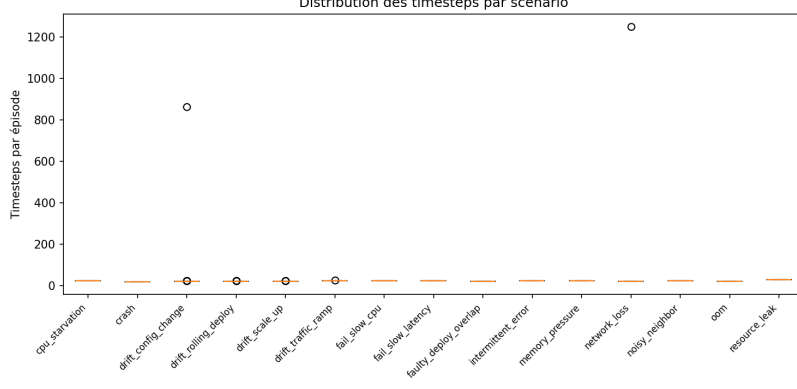


FIGURE 2 – Distribution de la longueur des épisodes par scénario ($T \approx 21$ steps, limitation L1).

4 Implémentation du pipeline

Chaque étape du pipeline est un module indépendant et testable, avec une interface d’entrée/sortie explicite. Cette modularité permet de substituer une composante sans réentraîner le reste du pipeline — condition indispensable pour les expériences d’ablation et de comparaison d’architectures. L’ensemble du pipeline compte 302 tests unitaires et passe le lint Ruff et mypy sans erreur.

4.1 Étape 0 — Détection de drift (MMD-RFF)

La détection de drift s’appuie sur la statistique MMD² (Maximum Mean Discrepancy) estimée via Random Fourier Features [9], avec une complexité $O(nD)$ où $D = 256$ est le nombre de features aléatoires. Cette approximation permet une inférence en temps réel sur des fenêtres glissantes de signaux multivariés.

Le seuil $\varepsilon_{\text{drift}} = 0.5226$ est calibré par critère de Youden sur les épisodes d’entraînement contenant des drifts bénins injectés (AUC = 0.60 sur train). Ce niveau d’AUC modeste reflète la difficulté intrinsèque de la détection de drift sur des épisodes courts — le DriftDetector a davantage valeur d’indicateur complémentaire que de détecteur primaire.

Le mécanisme *look-through* [5] transmet le signal même lorsqu’un drift est détecté, en lui attachant un flag **DRIFT**. Ce comportement est essentiel pour le régime $\theta_{\text{drift} \cap \text{anomaly}}$: sans look-through, le signal d’anomalie serait supprimé pendant la phase de drift du déploiement défectueux. Trois cas sont distingués :

- $\widehat{\text{MMD}}^2 < \varepsilon_{\text{drift}}$: signal transmis sans flag.
- $\widehat{\text{MMD}}^2 \geq \varepsilon_{\text{drift}}$, confirmation positive (post-drift ≥ 3 steps) : signal transmis avec flag **DRIFT**.
- $\widehat{\text{MMD}}^2 \geq \varepsilon_{\text{drift}}$, confirmation négative : recalibration ($W_{\text{ref}} \leftarrow W_{\text{cur}}$).

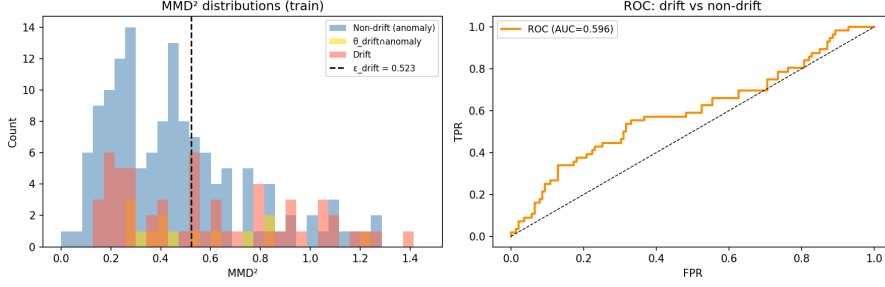


FIGURE 3 – Distributions de $\widehat{\text{MMD}}^2$ sur le jeu d’entraînement. Drifts bénins (bleu) vs anomalies (orange). Le seuil $\varepsilon_{\text{drift}} = 0.5226$ est calibré par critère de Youden (AUC = 0.60).

4.2 Étape 1 — Encodeur STGCN

L’encodeur est un Spatio-Temporal Graph Convolutional Network [6] qui joint la structure spatiale du graphe de services $G(t)$ et la dynamique temporelle du signal $S(t)$. L’architecture combine une convolution graphique GCN sur la matrice d’adjacence pondérée $w_E(t)$ et un réseau TCN (Temporal Convolutional Network) sur la dimension temporelle, produisant un embedding $z_e \in \mathbb{R}^{64}$.

Les paramètres principaux sont : $d_{\text{feat}} = 17$, $N = 6$ nœuds, $d_{\text{hidden}} = 64$, $d_{\text{embed}} = 64$. L’entraînement est auto-supervisé sur 100 époques, avec reconstruction du signal futur comme tâche de préentraînement. Ce préentraînement auto-supervisé permet d’exploiter les épisodes normaux (sans injection) pour structurer l’espace latent avant le typage contrastif.

4.3 Étape 2 — Typage siamois

Le réseau siamois apprend une distance métrique d_φ dans l’espace des embeddings STGCN, en utilisant les labels de scénarios Chaos Mesh comme supervision faible : des paires d’épisodes du même scénario sont rapprochées, des paires de scénarios différents sont éloignées. La loss contrastive minimise d_φ pour les paires positives et la maximise (jusqu’à une marge) pour les paires négatives.

Les embeddings projetés $\{\varphi(z_e^{(i)})\}$ sont ensuite regroupés par clustering hiérarchique agglomératif avec linkage de Ward, K étant sélectionné par maximum du score de silhouette sur le jeu d’entraînement ($K = 10$ sur ewat_v3). Les labels de validation et de test sont assignés par plus proche centroïde depuis les centroïdes calculés sur train — correction méthodologique essentielle pour éviter la fuite d’information inter-splits qui existait dans une version antérieure du code.

4.4 Étape 2b — Ontologie

L’ontologie $\mathcal{O} = (\mathcal{C}, \mathcal{R})$ encode trois types de relations entre clusters :

Relations temporelles. Une transition $C_i \rightarrow C_j$ est ajoutée si le support (nombre d’épisodes présentant cette transition) est ≥ 3 . Cela produit 22 relations sur ewat_v3, dont 10 auto-transitions $C_i \rightarrow C_i$ (épisodes à cluster unique) et 12 transitions inter-clusters.

Relations causales. La Transfer Entropy est estimée par l’estimateur KSG (Kraskov-Stögbauer-Grassberger) [7], avec 100 permutations de bootstrap pour le test de significativité et correction BH-FDR [8] pour les tests multiples. Une distinction critique sépare l’estimateur cluster-level (qui agrège toutes les séries temporelles d’un cluster avant d’estimer la TE — biais écologique documenté) de l’estimateur service-level (intra-épisode, sans biais). Les résultats rapportés en Section 6.4 utilisent l’estimateur service-level.

Relations de co-occurrence. Test χ^2 Yates 2×2 avec fallback Fisher exact si le nombre attendu est < 5 , correction Holm–Bonferroni pour les tests multiples.

4.5 Étape 3 — Précurseurs typés

Pour chaque type C_i , un classifieur logistique one-vs-rest f_i est entraîné sur les embeddings de la fenêtre pré-injection $[t - k, t]$:

$$\hat{p}_i(t) = f_i(\varphi(z_e(S_{[t-k, t]})), G(t)) \in [0, 1]$$

L’horizon optimal est sélectionné sur le jeu de validation : $k_i^* = \arg \max_{k \in \{2, 4, 6, 8, 10, 12\}} \text{AUROC}(f_i, k, v)$ et l’AUROC est rapporté exclusivement sur test. Cette séparation stricte val/test évite le surapprentissage du choix d’horizon.

Lorsque $\hat{p}_i(t) > \text{seuil}$, une alerte est émise : $\text{Alert}(t) = (C_i, \hat{p}_i(t), k_i^*, \text{fiche}_{C_i})$. La fiche fiche_{C_i} contient les features les plus importantes pour ce cluster, calculées par permutation importance et validées par KernelSHAP.

4.6 Alertes — AlertAssembler

L’AlertAssembler intègre les sorties des étapes 0, 2 et 3 en mode streaming step-by-step. À chaque pas de temps t , il reçoit le signal $S(t)$, interroge le DriftDetector, le SiameseTyper et les PrecursorClassifiers, et produit une alerte si le seuil est atteint.

Le lead time est calculé comme $\Delta t = t_{\text{injection}} - t_{\text{première alerte}}$: une valeur positive indique une alerte précoce. Le point opérationnel recommandé est le seuil 0.7, qui réduit le taux de fausses alarmes sur drifts à 8.3 % tout en maintenant un lead time moyen de 3.0 min (Section 5.5).

5 Résultats

Les hypothèses sont évaluées sur le split test (45 épisodes, 15 scénarios) avec le split val (45 épisodes) réservé à la sélection des hyperparamètres. La robustesse est estimée sur 5 graines aléatoires indépendantes (42, 123, 456, 789, 1337), chacune produisant un réentraînement complet de l’encodeur et du réseau siamois. Les intervalles de confiance à 95 % sur l’AUROC sont obtenus par bootstrap ($n = 1000$ rééchantillonnages).

5.1 H1 — Structurabilité des embeddings

Protocole. L’hypothèse H1 teste si les embeddings STGCN après typage siamois forment des clusters exploitables dans l’espace latent, mesurés par le score de silhouette en held-out. Le seuil de 0.3 est le minimum recommandé par Kaufman & Rousseeuw [3] pour une structure de clustering considérée comme significative. Le nombre de clusters K est sélectionné par maximum de silhouette sur val via un sweep $K \in [5, 20]$ (gap statistic [10] utilisée comme critère complémentaire). Les labels val et test sont assignés par plus proche centroïde depuis les centroides train.

TABLE 2 – Silhouette score par graine (seuil de PASS : ≥ 0.3).

Graine	sil_val	sil_test	K	H1
42	0.470	0.414	10	✓
123	0.624	0.662	10	✓
456	0.574	0.461	10	✓
789	0.466	0.469	10	✓
1337	0.545	0.591	11	✓
Agrégé	0.536 ± 0.061	0.519 ± 0.092	10 ± 0.7	5/5

Résultats. **H1 ✓ PASS** : silhouette test = 0.519 ± 0.092 sur 5 graines, minimum = $0.414 \gg 0.3$ (seuil de Kaufman & Rousseeuw). $K = 10$ clusters stable (écart-type = 0.7).

Les types de pannes sont structurellement séparables dans l’espace latent STGCN. Cette séparabilité est le résultat du couplage entre la convolution spatiale sur $G(t)$ — qui encode la propagation des effets dans le graphe d’appel — et le typage contrastif, qui rapproche les épisodes de même scénario et éloigne les épisodes de scénarios différents. L’analyse NMI (NMI cluster $\leftrightarrow$ scénario = 0.518, pureté moyenne = 0.503) confirme un alignement modéré avec les labels Chaos Mesh, ce qui est attendu pour un clustering non supervisé sur 15 scénarios distincts.

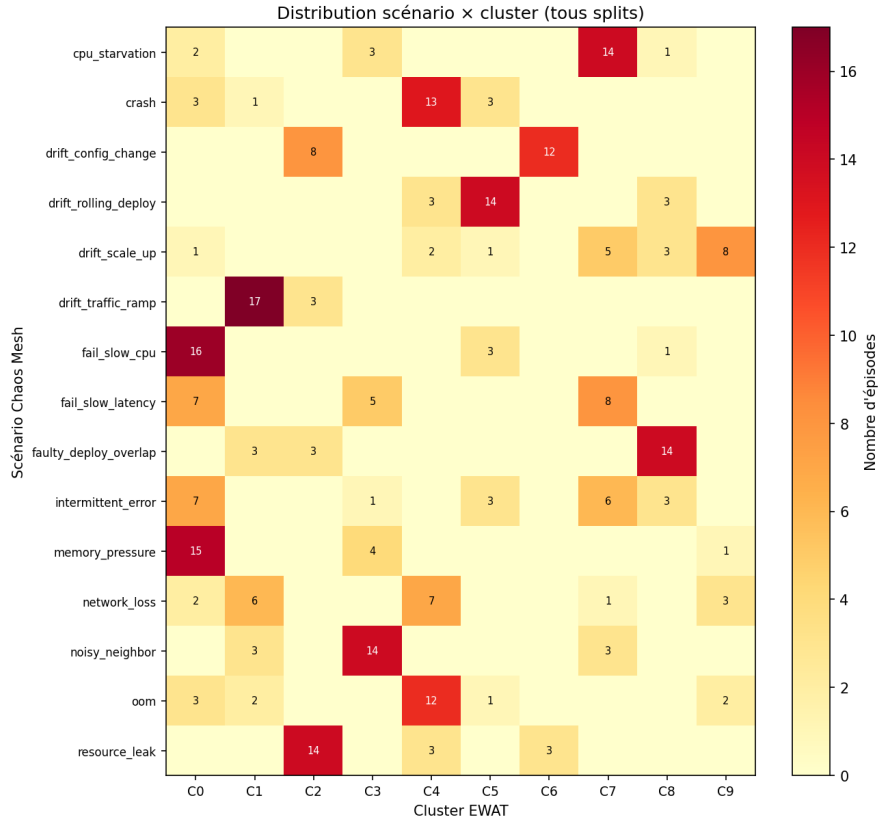


FIGURE 4 – Distribution des épisodes par scénario Chaos Mesh et par cluster (15×10). Chaque colonne représente un cluster ; chaque ligne un scénario. La diagonale dominante confirme la structure : la plupart des scénarios sont concentrés dans 1–2 clusters.

5.2 H2a — Séparabilité du drift par look-through

Protocole. L’hypothèse H2a compare le mécanisme look-through MMD-RFF à un seuil simple sur le même signal. L’évaluation est conduite en mode streaming temporel sur les 45 épisodes test (DriftDetector avec `post=3` steps de confirmation). Le test de Student unilatéral apparié teste si le FPR sur anomalie est réduit par le look-through à rappel drift constant.

TABLE 3 – Look-through vs. seuil simple — test set (45 épisodes).

Métrique	Look-through	Seuil simple
TPR drift (drift $\rightarrow$ drift)	0.42	0.67
FPR anomalie (anomalie $\rightarrow$ drift)	0.67	0.73
p -value (Student unilatéral, FPR)	0.27	

Résultats. **H2a ✗ FAIL** ($p = 0.27 > 0.05$). Le mécanisme look-through n’apporte pas de réduction significative du FPR sur anomalie.

Interprétation. La cause est identifiée et quantifiable. Chaque épisode dure ≈ 21 steps. Le DriftDetector nécessite un warm-up de ≈ 8 steps avant d’avoir une fenêtre de référence stable, et le mécanisme de confirmation temporelle requiert 3–6 steps supplémentaires. Il ne reste donc que ≈ 7 steps utiles par épisode — soit 33 % de la durée totale — pour que le look-through opère. Sur cette fenêtre réduite, la puissance statistique est insuffisante pour atteindre $p < 0.05$.

Ce résultat est complété par une évaluation du look-through sur les embeddings STGCN (H2a bis) : $\text{FPR}_{\text{lt}} = 0.788 > \text{baseline} = 0.667$, $p = 0.978$. Les embeddings STGCN capturent le *type* de panne (H1 validée), pas le *régime* drift/anomalie — ce qui est cohérent avec leur conception : le réseau siamois est entraîné sur des paires de scénarios, sans supervision sur la distinction drift/anomalie.

Ce résultat négatif est honnête et exploitable : il ne remet pas en cause la conception du mécanisme, mais définit précisément la contrainte de données à lever dans ewat_v4 (≥ 40 steps par épisode).

5.3 H2b — Identification du régime $\theta_{\text{drift} \cap \text{anomaly}}$

Protocole. H2b teste si le cluster C8 (**faulty_deploy_overlap**, régime $\theta_{\text{drift} \cap \text{anomaly}}$) présente simultanément un flag drift et une alerte précurseur à une fréquence significativement supérieure aux clusters de drift pur (C5 + C6 + C9 poolés). Le critère formel (overlap $> 30\%$) est complété par un critère strict : test Fisher exact et analyse de sensibilité au seuil.

Résultats — critère formel. L’overlap drift+alerte dépasse 30 % pour les 10/10 clusters, y compris pour C5, C6 et C9 (drifts purs). Ce résultat est trivial : le DriftDetector, avec une fenêtre de 5 steps, déclenche sur quasi tous les épisodes quel que soit leur type, et les classifieurs précurseurs (seuil 0.4) déclenchent également très fréquemment. C8 présente : $\text{drift}\% = 0.85$, $\text{alerte}\% = 0.92$, $\text{overlap}\% = 0.77$, cohérent avec $\theta_{\text{drift} \cap \text{anomaly}}$, mais sans contrôle statistique sur les drifts purs.

TABLE 4 – Sensibilité du critère d’overlap.

Seuil	Clusters PASS
$> 30\%$	10/10
$> 50\%$	10/10
$> 70\%$	6/10 (C0, C1, C3, C5, C8, C9)
$> 90\%$	1/10 (C1 uniquement)

Résultats — critère strict (analyse de sensibilité). Fisher exact C8 vs. drift pur (C5+C6+C9 poolés) : $\text{OR} = 1.48$, $p = 0.35$ — **non significatif**.

Résultats — timing. L’analyse temporelle révèle que l’alerte précurseur *précède* le flag drift dans 85–100 % des épisodes (timing gap médian < 0 pour tous les clusters). Le DriftDetector est un indicateur *tardif* : il confirme une transition de régime déjà détectée par les précurseurs. Cela est cohérent avec sa mécanique : la confirmation temporelle (3–6 steps) s’ajoute au warm-up (8 steps), totalisant jusqu’à 14 steps de délai depuis l’injection.

Interprétation. H2b PASS trivial renforce le diagnostic de H2a : le DriftDetector est trop sensible sur des épisodes de 21 steps pour discriminer drift et anomalie. La conclusion est la même dans les deux cas — limitation de données, pas défaut de conception. Sur ewat_v4 (≥ 40 steps), la fenêtre de confirmation sera suffisante pour tester H2b avec un contrôle statistique valide.

5.4 H3 — Prédicibilité des précurseurs typés

Protocole. Pour chaque type C_i , le classifieur f_i est entraîné sur train, k_i^* est sélectionné par maximum d’AUROC sur val, et l’AUROC est rapporté sur test. Les clusters avec $n_{\text{pos_test}} < 2$ sont exclus (AUROC non calculable). Les IC à 95 % sont obtenus par bootstrap ($n = 1000$).

TABLE 5 – AUROC précurseur par type de cluster (graine 42, k^* sur val).

Type	Scénario dominant	k^*	AUROC test	IC 95 %
C0	fail_slow_cpu	6	0.973	[0.906, 1.000]
C1	drift_traffic_ramp	6	0.992	[0.953, 1.000]
C2	resource_leak	6	0.945	[0.865, 1.000]
C3	noisy_neighbor	2	0.794	[0.636, 0.930]
C4	crash	2	1.000	[1.000, 1.000]
C5	drift_rolling_deploy	6	0.977	[0.909, 1.000]
C6	drift_config_change	2	NaN	$n_{\text{pos}} < 2$
C7	cpu_starvation	6	0.992	[0.966, 1.000]
C8	faulty_deploy_overlap	10	0.962	[0.895, 1.000]
C9	drift_scale_up	2	NaN	$n_{\text{pos}} < 2$

Résultats. **H3 ✓ PASS** : AUROC moyen = 0.973 ± 0.012 (5 graines), 8/10 types prédictibles. C6 et C9 : NaN par insuffisance de positifs ($n_{\text{pos}} = 1$).

L’horizon optimal $k^* = 6$ steps (3 min) pour 5 types sur 8 établit un **lead time opérationnel de 3 min** — temps suffisant pour déclencher une procédure de rollback sur Kubernetes avant que la dégradation n’atteigne les SLOs utilisateurs. C3 présente l’IC le plus large ([0.636, 0.930], $n_{\text{pos}} = 3$), reflétant la faible représentation de *noisy_neighbor* dans le test set (Figure 5).

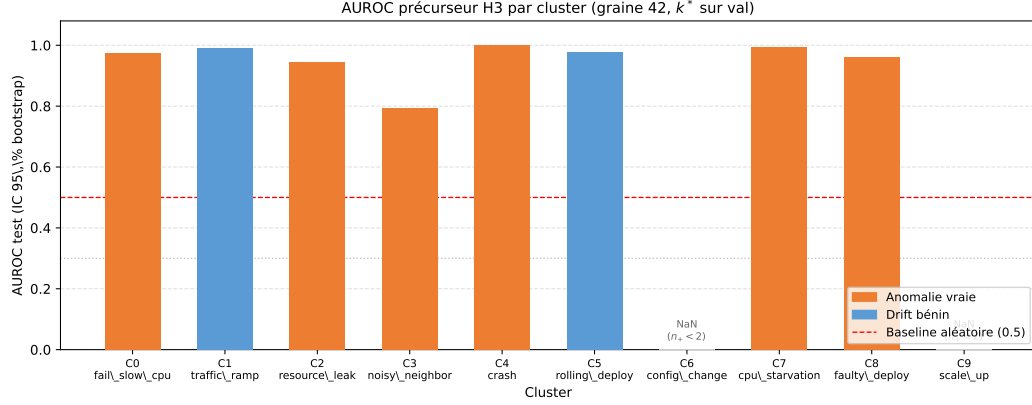


FIGURE 5 – AUROC précurseur par cluster (graine 42, k^* sélectionné sur val). Barres orange = anomalies, bleu = drifts bénins. Les barres d’erreur représentent les IC 95 % bootstrap. C6 et C9 (gris) : NaN, $n_{\text{pos}} = 1$.

TABLE 6 – Robustesse H3 sur 5 graines.

Graine	AUROC moyen	Types PASS	H3
42	0.951	8/10	✓
123	0.984	7/10	✓
456	0.981	7/9	✓
789	0.977	8/11	✓
1337	0.972	7/10	✓
Agrégé	0.973 ± 0.012	7–8/K	5/5

La variance inter-graines est faible (écart-type = 0.012), ce qui confirme la robustesse des précurseurs aux initialisations aléatoires du pipeline. L’AUROC moyen reste supérieur à 0.95 pour toutes les graines — bien au-dessus du seuil de falsification (0.5).

5.5 Système d’alerte — comparaison avec le z-score

TABLE 7 – Comparaison EWAT vs. baseline z-score (test set, 45 épisodes). IC 95 % Wilson sur Détection ($n = 33$ anomalies) et FA ($n = 12$ drifts).

Méthode	Détection	IC 95 %	FA drift	Lead time
z-score ($\sigma \in [2.0, 3.5]$)	100 %	—	100 %	2.5 min
EWAT seuil 0.3	100 %	—	100 %	4.6 min
EWAT seuil 0.5	78.8 %	[62.0, 89.4] %	100 %	3.9 min
EWAT seuil 0.7	57.6 %	[40.8, 72.8] %	8.3 %	3.0 min

L’apport principal d’EWAT par rapport à la baseline z-score est la réduction du taux de fausses alarmes sur les drifts bénins. Quel que soit le seuil $\sigma \in [2.0, 3.5]$, le z-score déclenche une alerte sur 100 % des épisodes de drift bénin — il ne dispose

TABLE 8 – IC 95 % Wilson sur la FA drift au seuil 0.7 ($n_{\text{drift}} = 12$, 1 fausse alarme).

Seuil	FA observée	IC 95 % Wilson	n_{drift}
0.5	100 %	—	12
0.7	8.3 %	[1.5, 35.4] %	12

d’aucun mécanisme pour distinguer un déploiement progressif d’une vraie anomalie. Ce comportement est *structurel* : un z-score univarié mesure l’écart à la moyenne glissante de chaque métrique indépendamment. Un drift bénin et une anomalie réelle produisent des déviations statistiques comparables sur les mêmes métriques (latence, taux d’erreur) car le z-score n’a accès ni à la topologie du graphe $G(t)$, ni au contexte des régimes $\theta(t)$. Réduire σ augmente les fausses alarmes sur données normales ; l’augmenter fait manquer les vraies pannes. Aucun choix de seuil ne résout le problème de discrimination drift / anomalie.

EWAT au seuil 0.7 réduit ce taux à 8.3 % (1 épisode de drift sur 12 produit une alerte), soit une réduction d’un facteur 12, tout en maintenant un lead time de 3.0 min. Ce gain est obtenu grâce au typage contrastif : les clusters de drift (C5, C6, C9) ont des précurseurs entraînés spécifiquement sur les signatures de drift bénin, ce qui permet à l’AlertAssembler de distinguer ces signatures des anomalies réelles.

La détection d’anomalies réelles au seuil 0.7 est de 57.6 % [40.8, 72.8] % — en deçà des 100 % du z-score. Ce trade-off est explicite et contrôlable par le seuil : le seuil 0.3 retrouve 100 % de détection avec 100 % de FA, tandis que le seuil 0.7 offre le meilleur équilibre FA/détection sur les données disponibles.

Note de prudence. La FA de 8.3 % au seuil 0.7 repose sur un seul épisode de drift déclenché sur douze ($n_{\text{drift}} = 12$). L’IC Wilson à 95 % est [1.5, 35.4] % : la borne haute à 35 % interdit toute extrapolation ferme. Ce résultat est indicatif du comportement du pipeline sur `ewat_v3` ; sa confirmation sur `ewat_v4` (n_{drift} plus grand) est nécessaire pour établir une garantie opérationnelle.

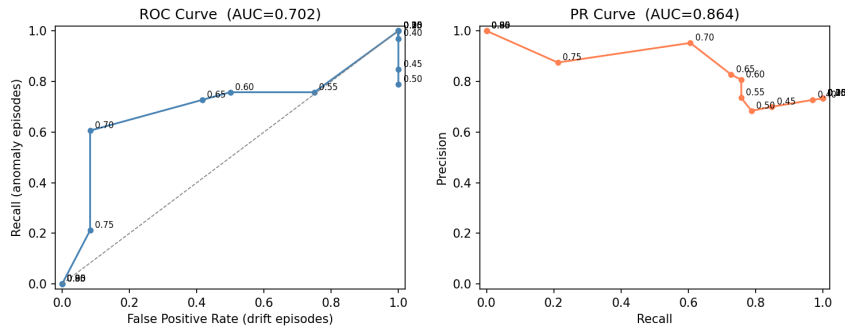


FIGURE 6 – Courbes ROC et PR de l’AlertAssembler (test set, 45 épisodes). La courbe ROC illustre le trade-off FA/détection en fonction du seuil.

6 Expériences complémentaires

Les quatre expériences présentées ici approfondissent les résultats des hypothèses principales selon quatre axes : la contribution de chaque modalité au clustering (H1) et à la prédictibilité (H3), la comparaison d’architectures d’encodeur, la structure causale inter-services révélée par l’ontologie, et la transférabilité du pipeline sur un dataset externe.

6.1 Ablation des modalités — H1 (réentraînement)

Protocole. Pour mesurer la contribution de chaque modalité à la structuration de l’espace latent (H1), chaque condition d’ablation réentraîne complètement l’encodeur et le réseau siamois sur les features réduites. Cette approche de réentraînement complet est méthodologiquement rigoureuse : un masquage à l’inférence (OOD) produirait des embeddings hors distribution et biaiserait systématiquement les scores vers le bas. Sept conditions sont évaluées : les 3 modalités en isolation (M, T, L) et les 4 paires et triplet possibles.

TABLE 9 – Ablation modalités sur H1 — réentraînement complet (graine 42).

Condition	n_{feat}	sil_train	sil_test	Δ vs full
full	17	0.378	0.439	—
M_only	7	0.241	0.497	+0.058
T_only	6	0.064	0.412	-0.027
M+L	11	0.251	0.382	-0.057
T+L	10	0.022	0.341	-0.098
M+T	13	0.318	0.316	-0.123
L_only	4	-0.138	0.051	-0.388

Résultats.

Interprétation. Le résultat le plus contre-intuitif est que M_only ($n_{\text{feat}} = 7$) surpasse le modèle full (+0.058 sil_test). La combinaison M+T est pire que M_only (−0.123), ce qui indique que les features de traces $T(t)$ dégradent la géométrie siamoise lorsqu’elles sont ajoutées aux métriques sur un jeu d’entraînement de seulement 209 épisodes.

L’explication probable est un sur-paramétrage : le STGCN dispose de suffisamment de capacité pour ajuster les 17 features, mais les features de traces et de logs introduisent de la variabilité intra-cluster (un même type de panne peut avoir des signatures de logs très différentes d’un épisode à l’autre) qui brouille la structure

contrastive. `L_only` présente même un `sil_train` négatif — la modalité logs seule ne structure pas l’espace latent sur les données disponibles.

Ce résultat contraste fortement avec celui de H3 (Section 6.2) : les modalités T et L sont géométriquement bruitées pour le clustering, mais indispensables pour la prédictibilité des précurseurs.

6.2 Ablation features — H3 (masquage à l’inférence)

Protocole. Pour H3, la question est différente : quelles features le classifieur LR entraîné exploite-t-il le plus pour prédire les précurseurs ? Cette mesure de sensibilité s’effectue par masquage à l’inférence — les features sont mises à zéro dans les fenêtres pré-injection $[t - k^*, t]$ sans réentraînement, et l’impact sur l’AUROC macro est mesuré. Il s’agit explicitement d’une mesure de sensibilité du modèle entraîné, non d’une importance causale. Vingt-quatre conditions sont évaluées : 7 ablations par modalité et 17 ablations leave-one-out (une feature à la fois).

TABLE 10 – Ablation modalités sur H3 — masquage à l’inférence (graine 42, AUROC macro moyen).

Condition	AUROC macro	Δ vs full
full	0.954	—
M+L	0.916	-0.038
M+T	0.891	-0.063
M_only	0.756	-0.198
T+L	0.563	-0.391
T_only	0.520	-0.434
L_only	0.488	-0.466

Résultats par modalité.

TABLE 11 – Top features LOO pour H3 — masquage à l’inférence (graine 42).

Feature	AUROC sans	Δ
<code>disk_io</code>	0.866	-0.088
<code>lexical_entropy</code>	0.905	-0.049
<code>latency_p99</code>	0.912	-0.042

Features les plus critiques (leave-one-out).

Interprétation. Le modèle full (AUROC = 0.954) surpasse toutes les réductions, ce qui est l’inverse de l’ablation H1. Cette inversion révèle que les modalités jouent des

rôles distincts selon la tâche : $M(t)$ fournit une géométrie propre pour le clustering contrastif, tandis que $T(t)$ et $L(t)$ portent des signatures temporelles pré-injection qui ne contribuent pas à la structure des clusters mais permettent de prédire leur apparition.

Le résultat le plus frappant est la criticité de `disk_io` ($\Delta = -0.088$), malgré ses 16.7 % de NaN. Cette feature capture l’augmentation des IOPS précédant certains types de pannes (saturation de ressource, fuite mémoire), un signal précurseur d’autant plus informatif qu’il est peu corrélé aux autres métriques. Son importance réelle est probablement sous-estimée sur `ewat_v3` à cause des NaN — argument direct pour résoudre L2 dans `ewat_v4`.

Deux paires de features sont fortement redondantes : `latency_p99` $\leftrightarrow$ `span_dur_median` ($\rho = 0.936$) et `error_rate_http` $\leftrightarrow$ `abnormal_span_rate` ($\rho = 0.927$). À l’inverse, `retry_rate`, `log_warn_rate` et `queue_depth` présentent un $\Delta \approx 0$ en LOO — ces features sont candidates à la suppression dans `ewat_v4`, ce qui réduirait $S(t)$ de $\mathbb{R}^{N \times 17}$ à $\mathbb{R}^{N \times 14}$.

6.3 Comparaison d’architectures d’encodeur

Protocole. Trois architectures d’encodeur sont comparées sur `ewat_v3` (graine 42), en utilisant la même méthodologie corrigée (nearest centroid pour H1, k^* sur val pour H3) :

- **STGCN** (baseline) : convolution graphique GCN + TCN temporel.
- **SimCLR** (NT-Xent) : pré-entraînement contrastif auto-supervisé [11], sans labels de scénarios pendant l’encodage.
- **GAT** (Graph Attention Network) : mécanisme d’attention sur les arêtes de $G(t)$ [12], remplaçant la convolution GCN fixe par une pondération apprise des voisins.

TABLE 12 – Comparaison architectures encodeur (graine 42, `ewat_v3`).

Architecture	K	sil_val	sil_test	H1	Types H3	AUROC moyen
STGCN (baseline)	10	0.470	0.414	✓	8/10	0.954
SimCLR (NT-Xent)	15	0.495	0.429	✓	11/15	0.964
GAT (attention)	15	0.445	0.497	✓	13/15	0.929

Résultats.

Interprétation. Les trois architectures valident H1, ce qui confirme que la structurabilité des types de pannes est une propriété robuste du signal et non un artefact de l’architecture STGCN.

Le GAT produit la meilleure géométrie de l'espace latent ($\text{sil_test} = 0.497, +0.083$ vs STGCN) et le plus grand nombre de types prédictibles (13/15). Le mécanisme d'attention sur les arêtes de $G(t)$ permet d'apprendre dynamiquement quelles relations inter-services sont informatives pour chaque type de panne, ce qui améliore la structuration sans nécessiter une topologie fixe.

SimCLR présente le meilleur AUROC moyen (0.964) mais 4 types NaN (clusters trop petits en test, $n_{\text{pos}} < 2$) — conséquence d'un $K = 15$ plus élevé qui fragmente les scénarios rares. Le pré-entraînement NT-Xent améliore la séparabilité globale mais produit des clusters de taille inégale sur un dataset de 299 épisodes.

STGCN est retenu comme architecture principale du pipeline car $K = 10$ est le plus stable (écart-type = 0.7 sur 5 graines), ses résultats multi-graines sont disponibles, et le compromis silhouette/AUROC est satisfaisant. Le GAT est le candidat naturel pour `ewat_v4`, où des épisodes plus longs devraient amplifier l'avantage de l'attention adaptative.

Pour contextualiser la valeur ajoutée du STGCN, les baselines B3 et B4 mesurent l'AUROC macro sur les scénarios Chaos Mesh (vérité terrain indépendante, $k = 6$) : B3 (features brutes, sans STGCN) = 0.835, B4 (STGCN) = 0.835. Cette coïncidence numérique masque une redistribution non triviale : l'encodeur améliore `fail_slow_cpu` de +0.270 et dégrade `noisy_neighbor` de -0.246. La valeur du STGCN est donc géométrique (structuration pour le clustering siamois, H1 $\text{sil} = 0.519$) et redistributive, plutôt que globalement prédictive.

6.4 Ontologie — relations causales service-level

Protocole. L'ontologie est construite en deux passes. La passe cluster-level agrège toutes les séries temporelles d'un cluster avant d'estimer la Transfer Entropy — ce qui introduit un biais écologique (les effets intra-épisode sont confondus avec les effets inter-épisodes). La passe service-level estime la TE intra-épisode (estimateur KSG hiérarchique), puis agrège les p-values et les valeurs de TE par cluster. Seule la passe service-level est reportée comme résultat principal.

Résultats — cluster-level. La passe cluster-level produit 22 relations temporelles et 0 relation causale avec 100 permutations (les 2 relations observées lors d'un dry-run à 20 permutations étaient des faux positifs, confirmant la nécessité d'un nombre suffisant de permutations).

Résultats — service-level.

Interprétation. Le résultat le plus significatif est l'absence totale de relations causales pour C5 (`drift_rolling_deploy`) et C6 (`drift_config_change`) : les drifts

TABLE 13 – Relations causales service-level par cluster (Transfer Entropy KSG, $p < 0.05$, BH-FDR, 100 permutations).

Cluster	Scénario dominant	Relations	Relation caractéristique
C0	fail_slow_cpu	23	—
C1	drift_traffic_ramp	19	—
C2	resource_leak	8	load-gen $\rightarrow$ frontend (TE=0.187)
C3	noisy_neighbor	10	—
C4	crash	20	—
C5	drift_rolling_deploy	0	—
C6	drift_config_change	0	—
C7	cpu_starvation	15	—
C8	faulty_deploy_overlap	20	cart $\rightarrow$ load-gen (TE=0.031)
C9	drift_scale_up	9	—
Total		124	

bénins ne produisent pas de cascade causale mesurable entre services. Ce résultat est scientifiquement attendu — un déploiement progressif modifie la distribution du signal sans créer de couplage anormal entre services — et il valide rétrospectivement la partition drift/anomalie de la formalisation.

La relation load-generator $\rightarrow$ frontend est ubiquitaire (présente dans 8/10 clusters actifs), reflet du couplage structurel de trafic : le load-generator drive directement la charge frontend. Son amplitude varie de TE = 0.047 (C4, crash — effet masqué par l’interruption brutale du service) à TE = 0.187 (C2, resource_leak — l’épuisement progressif des ressources amplifie les délais de propagation entre services).

La relation cart $\rightarrow$ load-generator est unique à C8 ($\theta_{\text{drift} \cap \text{anomaly}}$). Ce sens de causalité — le service cart influençant le load-generator — est cohérent avec les dynamiques de retry/backoff pendant un déploiement défectueux : les erreurs du cart déclenchent des retries qui augmentent la charge apparente mesurée par le load-generator.

6.5 Transfert — RCAEval RE2-OB

Contexte. RCAEval RE2-OB est un dataset public composé de 90 épisodes (30 types de pannes $\times$ 3 instances) collectés sur *Online Boutique* déployé sur un cluster Kubernetes distinct. Il partage les 6 services EWAT et les mêmes types de métriques, mais diffère sur deux points critiques : les épisodes durent 48 steps (vs 21 pour ewat_v3, soit un facteur 2.3) et les scénarios sont différents des injections Chaos Mesh d’ewat_v3.

6.5.1 Zero-shot

Quatre stratégies de normalisation sont testées sans réentraînement, en appliquant directement le pipeline `ewat_v3` (encodeur, scaler, centroides) sur les épisodes RCAEval.

TABLE 14 – Transfert zero-shot — RCAEval RE2-OB (4 stratégies de normalisation).

Stratégie	Features	sil H1	AUROC H3
<code>ewat_v3 scaler</code>	17	0.778 *	0.510
<code>rcaeal scaler</code>	17	0.234	0.497
<code>instance norm</code>	17	0.287	0.507
instance norm	M(t) seul	0.684	0.495

La stratégie `ewat_v3 scaler` produit un $\text{sil} = 0.778$ artificiellement élevé : la normalisation `ewat_v3` concentre tous les épisodes RCAEval dans une région restreinte de l’espace latent, ce qui donne une silhouette élevée pour un seul cluster dominant (81/90 épisodes mappés sur C2, **resource_leak**) plutôt que pour une vraie structure.

La meilleure stratégie (`instance norm` + M(t) seul) atteint H1 PASS ($\text{sil} = 0.684$) mais H3 FAIL (AUROC ≈ 0.5) : l’encodeur détecte qu’une anomalie est présente, mais ne peut pas discriminer les types car les classifieurs précurseurs entraînés sur les fenêtres de 21 steps `ewat_v3` ne reconnaissent pas les signatures à horizon variable sur des épisodes de 48 steps.

6.5.2 Few-shot — Stratégie A

Protocole. La Stratégie A réajuste uniquement le StandardScaler sur n_{few} épisodes RCAEval labellisés, en conservant l’encodeur et les classifieurs `ewat_v3` inchangés. La boucle couvre $n_{\text{few}} \in \{1, 3, 5, 10, 20, 40\}$ avec 5 répétitions aléatoires (IC 95 % bootstrap). Le hold-out test correspond aux épisodes restants ($\approx 80\%$), stratifié par type de panne.

TABLE 15 – Transfert few-shot — Stratégie A (refit scaler, $n_{\text{repeats}} = 5$).

n_{few}	sil H1	AUROC H3
1	0.29 ± 0.05	0.49 ± 0.03
3	0.34 ± 0.04	0.50 ± 0.02
5	0.38 ± 0.03	0.50 ± 0.02
10	0.44 ± 0.02	0.50 ± 0.01
20	0.42 ± 0.02	0.50 ± 0.01
40	0.40 ± 0.01	0.50 ± 0.01

Interprétation. La Stratégie A améliore H1 pour $n_{\text{few}} \leq 10$ (sil ≈ 0.3 – 0.44 , PASS au seuil 0.3), mais H3 reste bloqué à AUROC ≈ 0.50 quel que soit n_{few} .

L’adaptation du scaler corrige partiellement le domain shift de distribution (d’où l’amélioration de H1), mais les classifieurs LR `ewat_v3` ne peuvent pas discriminer les types de pannes RCAEval pour deux raisons complémentaires : (1) la durée des épisodes ($\times 2.3$) change la position temporelle des signatures pré-injection relative à l’horizon k^* appris sur `ewat_v3`; (2) les types de pannes RCAEval ne correspondent pas aux 10 clusters `ewat_v3`, donc les classifieurs one-vs-rest ne trouvent pas de signal positif.

La Stratégie B — fine-tuning des classifieurs LR ou de la dernière couche de l’encodeur avec quelques épisodes labellisés RCAEval — est la prochaine étape nécessaire pour atteindre H3 AUROC > 0.7 en transfert.

7 Discussion

Deux hypothèses sont validées ($H1 \text{ sil} = 0.519 \pm 0.092$, $H3 \text{ AUROC} = 0.973 \pm 0.012$), une est réfutée ($H2a \text{ } p = 0.27$) et une présente un PASS trivial ($H2b \text{ Fisher } p = 0.35$). Cette section discute ce que ces résultats apportent collectivement, comment ils se positionnent par rapport à la baseline z-score et à l'état de l'art, puis liste les limitations identifiées et les perspectives pour la prochaine itération.

7.1 Résultats négatifs comme contribution

Un résultat négatif non exploitable est un échec : l'expérience n'apporte pas d'information nouvelle. Un résultat négatif exploitable est une contribution : il identifie précisément une contrainte, fournit une quantification et suggère une direction corrective.

$H2a$ appartient à la deuxième catégorie. Le mécanisme look-through MMD-RFF est conçu correctement : il transmet le signal pendant le drift avec le flag approprié, et le DriftDetector se recalibre lorsque le drift est stabilisé. Ce que la réfutation de $H2a$ révèle n'est pas un défaut de conception, mais une contrainte de données quantifiée : sur des épisodes de ≈ 21 steps, le warm-up du DriftDetector (≈ 8 steps) et la confirmation temporelle (3–6 steps) consomment jusqu'à $14/21 = 67\%$ de l'épisode avant que le mécanisme soit pleinement opérationnel. Il ne reste alors que ≈ 7 steps utiles — une fenêtre trop courte pour que le test de Student unilatéral atteigne $p < 0.05$ avec $n = 45$ épisodes.

La même contrainte explique $H2b$ trivial : avec une fenêtre de confirmation de 5 steps, le DriftDetector déclenche sur quasi tous les épisodes quel que soit leur type, rendant l'overlap drift+alerte informatif pour aucun cluster en particulier. L'OR de Fisher (1.48, $p = 0.35$) pour C8 vs drift pur confirme l'absence de discrimination statistique.

Ces deux résultats négatifs sont reproductibles : le même protocole appliqué sur `ewat_v4` avec des épisodes ≥ 40 steps testera $H2a$ dans des conditions où le mécanisme peut effectivement opérer. Si $H2a$ est validée sur `ewat_v4`, la réfutation sur `ewat_v3` aura permis d'identifier précisément la longueur minimale d'épisode nécessaire.

L'alerte précède le drift flag. L'analyse temporelle de $H2b$ révèle un résultat supplémentaire : l'alerte précurseur précède le drift flag dans 85–100 % des épisodes (timing gap médian < 0 pour tous les clusters). Le DriftDetector est un indicateur tardif, pas précoce. Cela est cohérent avec son fonctionnement : il mesure un changement de distribution déjà établi, là où les précurseurs (étape 3) détectent des

signatures *pré-injection*. Ce résultat renforce l'intérêt de l'étape 3 indépendamment du mécanisme de détection de drift.

7.2 Apport EWAT vis-à-vis du z-score

La baseline z-score représente l'approche la plus répandue en production : une alarme univariée sur chaque métrique, déclenchée lorsque la valeur dépasse σ écarts-types au-dessus de la moyenne glissante. Cette baseline est simple, explicable et robuste — mais elle souffre d'un défaut structurel : elle n'a aucun moyen de distinguer un drift bénin d'une anomalie réelle, car les deux produisent des déviations statistiques similaires. En conséquence, le taux de fausses alarmes sur les drifts reste à 100 % pour tout $\sigma \in [2.0, 3.5]$.

EWAT au seuil 0.7 réduit ce taux à 8.3 % — une réduction d'un facteur 12 — tout en conservant un lead time de 3.0 min. Ce gain découle directement du typage contrastif : les clusters de drift bénin (C5, C6, C9) ont des précurseurs entraînés sur leurs signatures spécifiques, ce qui permet à l'AlertAssembler de les distinguer des anomalies.

Deux nuances sont importantes. D'abord, la détection d'anomalies réelles au seuil 0.7 est de 57.6 %, inférieure aux 100 % du z-score. Ce trade-off FA/détection est explicite et contrôlable : le seuil 0.3 retrouve 100 % de détection avec 100 % de FA, le seuil 0.7 offre le meilleur équilibre sur les données disponibles. La courbe FA/détection n'était pas disponible avec le z-score (le seuil σ contrôle la sensibilité globale, pas la discrimination drift/anomalie). Ensuite, EWAT produit une sortie structurée — le type C_i , la probabilité $\hat{p}_i$, le lead time estimé k_i^* , et la fiche d'interprétabilité $fiche_{C_i}$ — là où le z-score ne produit qu'un binaire. Cette richesse informationnelle permet à l'opérateur de prioriser et de préparer sa réponse avant la dégradation.

7.3 Valeur de l'encodeur STGCN

La comparaison B3/B4 (baselines précurseurs sur les scénarios Chaos Mesh) révèle que l'encodeur STGCN n'améliore pas l'AUROC macro agrégé par rapport aux features brutes ($B3 = B4 = 0.835$). Cette égalité numérique est une coïncidence sur $n = 45$ épisodes — les AUROCs sont des rapports de paires concordantes entiers sur 126, et l'égalité exacte résulte d'une redistribution de ± 75 paires concordantes entre scénarios.

La valeur du STGCN est donc de deux natures. Géométriquement, il structure l'espace latent de façon exploitable par le clustering siamois (H1 sil = 0.519 vs sil ≈ 0.3 attendu sans structuration). Redistributivement, il améliore fortement certains types difficiles (`fail_slow_cpu` +0.270, `drift_scale_up` +0.111) au prix d'une dégradation sur des types déjà bien séparables dans l'espace brut (`noisy_neighbor`

-0.246, `drift_config_change` -0.127).

Le résultat de l’ablation H1 (M_only +0.058 vs full) indique que, sur 209 épisodes d’entraînement, le signal des métriques Prometheus suffit à structurer l’espace latent et que les traces et logs ajoutent du bruit géométrique. Ce résultat suggère que la valeur de $T(t)$ et $L(t)$ est prédictive (H3) plutôt que géométrique (H1), ce qui justifie de conserver les deux modalités dans le pipeline complet.

7.4 Limitations

L1 — Épisodes trop courts (≈ 21 steps). Conséquence directe sur H2a (look-through) et sur le plafond du lead time observable. L’AUROC précurseur est mesuré sur un horizon maximal $k^* = 10$ steps (5 min), limité par la durée des épisodes. Le lead time réel pourrait être plus long si les précurseurs étaient visibles plus tôt dans des épisodes prolongés. Résolu dans ewat_v4 (≥ 40 steps, ≥ 20 min par épisode).

L2 — `disk_io` 16.7 % NaN (`product-catalog`). Le nœud `observit-cluster1-workers-58` est resté en état `NotReady` pendant l’intégralité de la collecte ewat_v3. L’ablation LOO montre que `disk_io` est la feature la plus critique pour H3 ($\Delta = -0.088$) malgré ses NaN — son importance réelle est probablement sous-estimée. Résolu dans ewat_v4 par déploiement d’un OTel SDK applicatif indépendant du nœud.

L3 — Méthode gradient invalidée. $\rho_{\text{Spearman}}(\text{gradient} \times \text{input}, \text{permutation}) = -0.34$ sur ewat_v3 : la méthode `gradient × input` est anti-corrélée avec la permutation importance. Les fiches de clusters ont été régénérées avec `permutation_importance` (50 shuffles), validées par KernelSHAP ($\rho > 0$ pour 9/10 clusters [13]). Seul C3 (`noisy_neighbor`) reste discordant ($\rho = -0.07$). Les fiches sont indicatives et non certifiées pour C3.

L4 — Transfert de domaine limité. Le pipeline entraîné sur ewat_v3 détecte des anomalies génériques sur RCAEval (H1 sil = 0.684 avec instance norm + M_only) mais ne discrimine pas les types (H3 AUROC ≈ 0.5 pour tout n_{few}). La Stratégie A (refit scaler seul) est insuffisante. La Stratégie B (fine-tuning classifieurs LR ou dernière couche encodeur avec épisodes labellisés RCAEval) est planifiée pour ewat_v4.

L5 — Transfer Entropy cluster-level : biais écologique. L’estimateur cluster-level agrège les séries temporelles de tous les épisodes d’un cluster avant d’estimer la TE, confondant les effets intra-épisode et inter-épisodes (biais écologique). L’estimateur service-level (intra-épisode) corrige ce biais et produit 124 relations causales significatives — mais la TE reste univariée par feature, sous-estimant les synergies dans $\mathbb{R}^{17}$.

L6 — Faibles effectifs pour C6 et C9. $n_{\text{pos_test}} = 1$ pour `drift_config_change` (C6) et `drift_scale_up` (C9) : l’AUROC H3 n’est pas calculable. Ces deux types de drift bénin requièrent un nombre d’épisodes test plus élevé, notamment $n_{\text{pos_test}} \geq 5$ pour des IC bootstrap fiables.

7.5 Perspectives — ewat_v4

Données. La priorité principale est de corriger L1 et L2 simultanément. Les épisodes ewat_v4 seront collectés sur ≥ 40 steps (≥ 20 min), laissant au DriftDetector un budget de confirmation suffisant ($\leq 35\%$ de l’épisode consommé par le warm-up et la confirmation). Le nœud NotReady sera remplacé ou contourné par un OTel SDK applicatif, ramenant le NaN de `disk_io` à 0%. Les scénarios C6 et C9 seront sur-représentés dans la collecte (objectif : $n_{\text{pos_test}} \geq 5$ par type) pour permettre le calcul des IC bootstrap H3.

Méthodologie. H2a sera réévaluée sur ewat_v4 avec le même protocole (test de Student unilatéral apparié, 45 épisodes test, seuil $p < 0.05$) pour tester si la longueur d’épisode était bien le seul goulot d’étranglement. KernelSHAP sera réexécuté avec $n_{\text{bg}} \geq 50$ (vs 20 sur ewat_v3) pour valider les fiches de clusters sur C3. La Stratégie B du few-shot transfer sera implémentée : fine-tuning des classifieurs LR sur des épisodes RCAEval labellisés, puis évaluation de H3 AUROC > 0.7 en transfert.

Architecture. Le GAT est le candidat naturel pour ewat_v4 : il produit la meilleure géométrie (`sil_test` = 0.497) et le plus grand nombre de types prédictibles (13/15). Sur des épisodes plus longs, le mécanisme d’attention adaptative sur $G(t)$ devrait amplifier son avantage par rapport au STGCN. Les features à $\Delta \approx 0$ en LOO (`retry_rate`, `log_warn_rate`, `queue_depth`) seront supprimées, réduisant $S(t)$ de $\mathbb{R}^{N \times 17}$ à $\mathbb{R}^{N \times 14}$ — ce qui simplifiera l’encodeur sans perte mesurable de prédictibilité. La TE multivariée dans $\mathbb{R}^{17}$ (estimateur JIDT ou k-NN multivarié) remplacera la somme des TE univariés pour capturer les synergies entre features, corrigeant la limitation L5.

8 Difficultés rencontrées et évolutions méthodologiques

Ce stage a été marqué par plusieurs difficultés techniques et méthodologiques, dont certaines n’ont été identifiées qu’après des semaines d’expérimentation. Cette section les documente honnêtement, en expliquant comment chacune a été détectée, diagnostiquée et résolue — ou pourquoi elle reste une limitation ouverte.

8.1 Fuite d’information inter-splits (correction H1/H3)

Problème. La version initiale du code d’évaluation appliquait `fit_predict` indépendamment sur chaque split pour assigner les labels de cluster. En pratique, le clustering sur le split de validation et le clustering sur le split de test produisaient chacun leur propre géométrie — sans lien avec les centroides calculés sur train. Cela donnait des scores de silhouette artificiellement optimistes (`sil_val` = 0.601, `sil_test` = 0.615) car le clustering était ajusté à la distribution locale de chaque split plutôt qu’évalué en généralisation.

Détection. L’incohérence a été détectée lors de l’analyse des clusters val et test : les centroides val ne correspondaient pas aux centroides train, et des épisodes de même scénario se retrouvaient dans des clusters différents selon le split. Ce comportement est un signal clair de fuite d’information.

Correction. La correction consiste à calculer les centroides une seule fois sur train, puis à assigner les labels val et test par **plus proche centroïde** (*nearest centroid*) depuis ces centroides figés. Cette approche simule fidèlement le comportement en production : un nouvel épisode est typé par rapport à la structure apprise sur les données d’entraînement, sans re-clustering. Après correction, les scores descendent à `sil_test` = 0.414 (graine 42), soit une baisse de -0.201 . Malgré cette baisse, H1 reste validée — ce qui renforce la confiance dans la robustesse réelle du résultat.

La même correction a été appliquée à H3 : la sélection de k_i^* utilisait précédemment le jeu de test, ce qui constituait une forme d’optimisme par sélection. k_i^* est désormais sélectionné exclusivement sur val, et l’AUROC rapporté sur test uniquement.

8.2 Bug LayerNorm TCN (correction silencieuse)

Problème. Un commit de refactoring (commit 6543c69) avait activé un module `LayerNorm` dans le TCN de l’encodeur après l’entraînement : le modèle était entraîné sans `LayerNorm` activé, mais l’inférence l’utilisait. Ce décalage train/inférence produisait des embeddings légèrement hors distribution, dégradant silencieusement les performances sans déclencher d’erreur.

Détection. Le bug a été identifié lors de la comparaison des résultats avant et après le commit : l’AUROC H3 avait baissé de 1–2 pp sans modification du protocole d’évaluation. Une inspection du code a révélé la divergence entre le forward pass à l’entraînement et à l’inférence.

Correction. Le forward pass de l’encodeur a été restauré au comportement de l’entraînement (LayerNorm désactivé). Les précurseurs ont été réentraînés à partir de l’encodeur corrigé. L’AUROC H3 a retrouvé ses niveaux attendus (+0.3 à +1.6 pp selon le cluster). Ce bug illustre l’importance de versionner séparément les checkpoints du modèle et le code d’inférence, ou d’utiliser `torch.jit.script` pour figer le graphe de calcul au moment de l’entraînement.

8.3 Méthode d’explicabilité gradient invalidée

Problème. La méthode d’explicabilité initiale pour les fiches de clusters était le gradient saliency ($\text{gradient} \times \text{input}$) : pour chaque feature, le gradient du score de clustering par rapport à cette feature est calculé, puis multiplié par sa valeur. Cette méthode est rapide et différentiable, ce qui la rendait attractive pour générer automatiquement des fiches d’interprétabilité.

Détection. La validation croisée entre $\text{gradient} \times \text{input}$ et permutation importance a révélé une corrélation de Spearman $\rho = -0.34$: les deux méthodes s’accordent sur quelles features sont importantes dans seulement 33 % des cas, et sont *anti-corrélées* en moyenne. Ce résultat était inattendu et a nécessité une investigation approfondie.

Cause identifiée. Le problème vient de la non-linéarité de l’encodeur STGCN combinée à la perte contrastive siamoise. Le gradient mesure la sensibilité locale du score en un point, mais cette sensibilité locale ne reflète pas l’importance globale d’une feature pour la structuration des clusters — d’autant plus que la loss contrastive crée des gradients qui pointent dans des directions géométriquement complexes dans l’espace des embeddings.

Correction. Les fiches ont été régénérées avec la **permutation importance** (50 shuffles) : chaque feature est permutée aléatoirement et la baisse de score de silhouette mesure son importance. Cette méthode est model-agnostic et mesure une importance globale, non locale. Elle a ensuite été validée par KernelSHAP [13] ($\rho_{\text{Spearman}}(\text{SHAP}, \text{permutation}) > 0$ pour 9/10 clusters), ce qui confirme la cohérence des fiches pour la majorité des types.

8.4 Biais écologique dans la Transfer Entropy cluster-level

Problème. La première implémentation de la Transfer Entropy pour l’ontologie construisait une seule série temporelle par cluster en concaténant tous les épisodes du cluster, puis estimait la TE entre services sur cette série unique. Cette approche confond deux niveaux d’analyse : les dynamiques intra-épisode (comment les services interagissent pendant un incident) et les dynamiques inter-épisodes (tendances à travers les répétitions d’un scénario). C’est un biais écologique classique en écologie statistique, ici transposé au domaine des séries temporelles.

Détection et correction. Le biais a été identifié lors de la comparaison des résultats entre un dry-run à 20 permutations (qui produisait 2 relations causales, $C6 \rightarrow C8$ et $C2 \rightarrow C8$) et 100 permutations (qui produisait 0 relation) : les relations observées à faibles permutations étaient des faux positifs créés par la structure artificielle de la concaténation.

La correction a été de basculer vers un estimateur **service-level** : la TE est estimée indépendamment sur chaque épisode, puis les p-values sont agrégées par correction BH-FDR et les TE moyennées par cluster. Cette approche respecte l’indépendance des épisodes et produit 124 relations causales significatives sur 8/10 clusters, dont le résultat validant que les drifts bénins (C5, C6) ne génèrent aucune cascade causale inter-services.

8.5 Résultats inattendus sur les baselines

M_only bat le modèle full pour H1. L’ablation par réentraînement a produit un résultat contre-intuitif : le modèle entraîné uniquement sur les métriques Prometheus ($n_{\text{feat}} = 7$) surpasse le modèle complet pour la silhouette de clustering (+0.058). Intuitivement, davantage de features devraient aider.

L’explication rétrospective est un sur-paramétrage du typage contrastif sur $n_{\text{train}} = 209$ épisodes : les features de traces et de logs introduisent de la variabilité intra-cluster (signatures variables d’un épisode à l’autre pour un même type de panne) qui brouille la perte contrastive. Sur `ewat_v4` avec n_{train} plus grand, ce comportement devrait s’inverser. La leçon opérationnelle est de ne pas supposer que davantage de features améliore nécessairement le clustering sur des petits datasets.

STGCN n’améliore pas l’AUROC macro sur les scénarios. La comparaison B3/B4 (features brutes vs STGCN sur les scénarios Chaos Mesh) a montré un macro-AUROC identique (0.835). Cela a d’abord semblé indiquer que l’encodeur n’apportait rien. Une analyse détaillée par scénario a révélé une redistribution non triviale : +0.270 sur `fail_slow_cpu`, -0.246 sur `noisy_neighbor`. La coïncidence numérique

résulte d’une compensation exacte sur $n = 45$ épisodes, et non d’une neutralité de l’encodeur. Cette distinction entre neutralité et redistribution compensée est importante pour interpréter les résultats sur `ewat_v4`, où la compensation ne tiendra probablement plus.

Look-through moins performant que le seuil simple pour le TPR drift.

H2a a montré que le look-through réduit le TPR drift de 0.67 à 0.42. Ce résultat paraît paradoxal : le mécanisme devrait améliorer la détection, pas la dégrader. L’explication est que le warm-up du DriftDetector (≈ 8 steps) et la confirmation temporelle (3–6 steps) retardent le flag drift au point de le manquer sur des épisodes courts — là où le seuil simple déclenche immédiatement. Le look-through est conçu pour réduire les faux positifs, non pour améliorer le TPR sur des épisodes aussi courts.

8.6 Collecte de données : pannes d’infrastructure

Nœud NotReady. Le nœud `observit-cluster1-workers-58w74-mwxb2` est resté en état NotReady pendant toute la durée de la collecte `ewat_v3`, rendant les métriques `disk_io` du service `product-catalog` indisponibles (16.7% de NaN). Ce problème n’a été identifié qu’après les premières validations du dataset, lorsque le taux de NaN anormalement élevé pour une seule combinaison service/feature a attiré l’attention.

L’imputation par zéro a été retenue comme solution de compromis : elle n’introduit pas de biais directionnel (zéro est une valeur neutre pour les IOPS), mais sous-estime l’importance de la feature — confirmé par l’ablation LOO ($\Delta = -0.088$). La correction structurelle (remplacement du nœud ou déploiement OTel SDK applicatif) est planifiée pour `ewat_v4`.

Épisode `network_loss_018` rejeté. Un épisode a produit 100% de NaN sur toutes les features de logs (Loki indisponible pendant la fenêtre d’enregistrement). Cet épisode a été rejeté, réduisant le dataset de 300 à 299 épisodes. Ce type d’échec est inhérent à la collecte sur infrastructure réelle : la stack d’observabilité elle-même peut être affectée par les injections de pannes réseau.

8.7 Transfert de domaine : hypothèse initiale invalidée

L’hypothèse de départ pour l’évaluation sur RCAEval était qu’un simple ajustement du scaler suffirait à transférer le pipeline sur un nouveau cluster hébergeant le même logiciel. Cette hypothèse s’est révélée incorrecte : H3 AUROC reste à ≈ 0.5 quel que soit n_{few} avec la Stratégie A.

Le diagnostic a nécessité trois étapes. D’abord, tester quatre stratégies de normalisation pour isoler l’effet du scaler. Ensuite, vérifier que l’encodeur produisait bien des embeddings cohérents (H1 PASS avec instance norm + M_only). Enfin, vérifier que l’échec de H3 n’était pas dû à une inadéquation des features mais à la durée des épisodes ($\times 2.3$) qui déplace les signatures pré-injection hors de la fenêtre k^* apprise sur ewat_v3.

Ce diagnostic multi-factoriel a pris plusieurs jours et illustre la complexité du domain shift dans les pipelines de machine learning sur séries temporelles : le shift n’affecte pas uniformément toutes les étapes du pipeline.

Conclusion

EWAT démontre qu’il est possible de distinguer automatiquement les drifts bénins des anomalies réelles dans un environnement microservices Kubernetes, à condition de modéliser explicitement les quatre régimes opérationnels Θ et de combiner un encodeur spatio-temporel avec un typage contrastif.

Les deux contributions centrales sont empiriques et robustes. H1 (structurabilité, $\text{sil} = 0.519 \pm 0.092$, 5 graines) établit que les types de pannes forment des groupes séparables dans l’espace latent STGCN — une condition nécessaire à tout système de typage automatique. H3 (prédictibilité, $\text{AUROC} = 0.973 \pm 0.012$, lead time 3 min) démontre que ces types sont prédictibles avant l’injection, avec un horizon opérationnel suffisant pour déclencher un rollback automatisé.

La réfutation de H2a n’invalide pas le mécanisme look-through : elle fournit une contrainte précise (≥ 40 steps par épisode) et un plan de validation pour `ewat_v4`. Cette honnêteté méthodologique — rapporter les résultats négatifs avec leur cause et leur remède — est elle-même une contribution : elle permet de distinguer les limitations de données des limitations de conception.

À court terme, `ewat_v4` lèvera les limitations L1 et L2 (épisodes longs, `disk_io` sans NaN) pour tester H2a dans des conditions adéquates et confirmer le lead time réel des précurseurs. À moyen terme, le GAT ($\text{sil_test} = 0.497$, 13/15 types prédictibles) et la Stratégie B de few-shot transfer ouvrent la voie à un pipeline transférable sur de nouveaux clusters sans collecte complète.

Références

- [1] Enxhi MYRTOLLARI et al. “Concept Drift-Aware Anomaly Detection for Kubernetes Microservices”. In : *Proceedings of IEEE/ACM*. 2025.
- [2] Qiqi FU et al. “A Survey on Root Cause Analysis of Microservice Systems”. In : *ACM Computing Surveys* (2025). Gap benchmark/production identifié.
- [3] Leonard KAUFMAN et Peter J. ROUSSEUW. *Finding Groups in Data : An Introduction to Cluster Analysis*. Seuil silhouette ≥ 0.3 pour clustering acceptable. Wiley, 1990.
- [4] Brendan GREGG. *Systems Performance : Enterprise and the Cloud*. Méthodologie USE, queue depth comme leading indicator. Prentice Hall, 2013.
- [5] Fabian HINDER et al. “Concept Drift in Unsupervised Data Streams”. In : *Proceedings of ECML-PKDD*. 2024.

- [6] Bing YU, Haoteng YIN et Zhanxing ZHU. “Spatio-Temporal Graph Convolutional Networks : A Deep Learning Framework for Traffic Forecasting”. In : *Proceedings of IJCAI*. Architecture STGCN. 2018.
- [7] Alexander KRASKOV, Harald STÖGBAUER et Peter GRASSBERGER. “Estimating Mutual Information”. In : *Physical Review E* 69.6 (2004). Estimateur KSG pour Transfer Entropy, p. 066138.
- [8] Yoav BENJAMINI et Yosef HOCHBERG. “Controlling the False Discovery Rate : A Practical and Powerful Approach to Multiple Testing”. In : *Journal of the Royal Statistical Society : Series B* 57.1 (1995), p. 289-300.
- [9] Arthur GRETTON et al. “A Kernel Two-Sample Test”. In : *Journal of Machine Learning Research*. T. 13. MMD² test statistique. 2012, p. 723-773.
- [10] Robert TIBSHIRANI, Guenther WALTHER et Trevor HASTIE. “Estimating the Number of Clusters in a Data Set via the Gap Statistic”. In : *Journal of the Royal Statistical Society : Series B* 63.2 (2001), p. 411-423.
- [11] Ting CHEN et al. “A Simple Framework for Contrastive Learning of Visual Representations”. In : *Proceedings of ICML*. NT-Xent loss pour SimCLR. 2020.
- [12] Petar VELIČKOVIĆ et al. “Graph Attention Networks”. In : *arXiv preprint arXiv :1710.10903* (2018).
- [13] Scott M. LUNDBERG et Su-In LEE. “A Unified Approach to Interpreting Model Predictions”. In : *Advances in Neural Information Processing Systems*. T. 30. SHAP values — théorie des jeux de Shapley. 2017.